Android Design

Componente Android (app BTLMARC): escucha los beacons, traduce cada anuncio a una `Medicion` y la guarda en el servidor a través de su lógica de cliente. Notación: Logical Design & Reverse Engineering Specification v3.

Component Design

La app Android escucha los anuncios iBeacon de la plaquita Arduino, se queda solo con el beacon del proyecto, los traduce a una `Medicion` y la sube por HTTP al servidor. Para no mezclar la pantalla con la red, el envío se hace a través de una **lógica de cliente** (`LogicaFakeApp`) que actúa como **proxy** del backend.

Clases y responsabilidades

Clase	Tipo	Responsabilidad
`MainActivity`	GUI	Pantalla principal; arranca/para el servicio
`ServicioEscuharBeacons`	Servicio BLE	Escucha, filtra y **throttlea** las tramas; delega el guardado
`AnalizadorBeacons`	Utilidad / lógica pura	Filtra la trama, traduce tipo/valor y monta JSON y URL
`Medicion`	Modelo	Datos de una medición (dominio)
`LogicaFakeApp`	Lógica de cliente (proxy)	Guarda la `Medicion` hablando con el servidor REST
`PeticionarioREST`	Comunicación	Envía un `POST`/`GET` JSON al endpoint del servidor
`TramaIBeacon`	Utilidad	Parsea el `byte[]` de un iBeacon (uuid, major, minor, txPower)
`Utilidades`	Utilidad	Conversiones de bytes/enteros

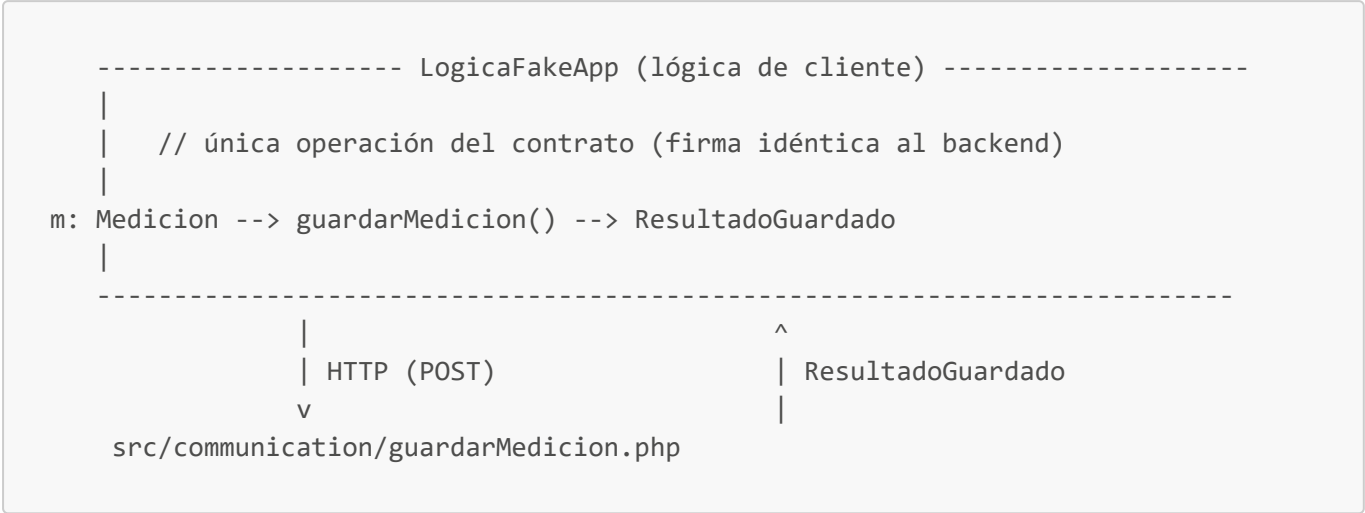
Tipos de dominio

```
Medicion = (
   id_plaquita: Text,   // "GTI-3A"
   tipo_medicion:  Text,   // "CO2" | "temperatura" | "prueba"
   valor_medicion: Z       // valor del proyecto (el minor de la trama, hoy 1234)
)

ResultadoGuardado = ( ok: B, error: Text | vacío, clase: ClaseError | vacío )
ClaseError        = { VALIDACION, PERSISTENCIA }
```

Lógica de cliente (proxy) — `LogicaFakeApp`

`LogicaFakeApp` es la **lógica fake del teléfono**: el equivalente móvil de `src/frontend_business_logic/` de la web. Expone a la app **un subconjunto** de la lógica de negocio del servidor (`business_logic_design.md`) con **firma idéntica**, pero implementado como **proxy**: por dentro monta el JSON, la URL y hace el `POST` al servidor en vez de tocar la base de datos.



Método (contrato)

Método	Tipo	Entrada	Salida
`guardarMedicion( m )`	mutante (`-->`)	`Medicion`	`ResultadoGuardado`

`Medicion` y `ResultadoGuardado` son los **mismos tipos de dominio** definidos en `business_logic_design.md`; la app no inventa tipos nuevos. Como la red es asíncrona, el `ResultadoGuardado` no se devuelve con `return`: se entrega por el callback `RespuestaApp.onResultado(...)`.

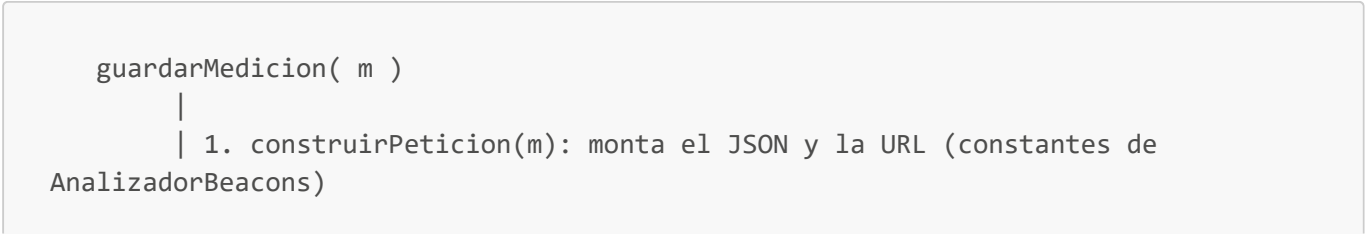
Identidad de firma con el backend

```
BACKEND  (business_logic_design.md):
   m: Medicion --> guardarMedicion() --> ResultadoGuardado

APP      (este documento):
   m: Medicion --> guardarMedicion() --> ResultadoGuardado   // MISMA firma
```

Por la **regla del subconjunto**, la app solo declara la operación que usa (el guardado); por la **regla del proxy**, la implementación llama al endpoint `src/communication/guardarMedicion.php` en vez de a la base de datos.

Flujo de datos (proxy)



```
        | 2. POST a src/communication/guardarMedicion.php vía PeticionarioREST
        | 3. interpretarRespuesta( codigo, cuerpo ):
        |       200 -> ResultadoGuardado( ok=true )
        |       400 -> ResultadoGuardado( ok=false, clase='validacion', error )
        |       500 -> ResultadoGuardado( ok=false, clase='persistencia', error )
        |       otro/red -> ResultadoGuardado( ok=false, clase='persistencia',
error )
        | 4. RespuestaApp.onResultado( resultado )
```

Diagrama de comportamiento (servicio)

```
   (BleScanner)
        |
        v   byte[] (ScanRecord)
   AnalizadorBeacons.motivoDescarte() --> Text     // filtro: ¿es nuestro beacon?
(null = sí)
        |
        v
   Medicion m = new Medicion( tib, nombre )
        |
        v
   AnalizadorBeacons.debeEnviarMedicion() --> B    // throttle: no repetir
demasiado seguido
        |
        | true
        v
   LogicaFakeApp.guardarMedicion( m ) --> ResultadoGuardado
        |
        v
   src/communication/guardarMedicion.php --> 200/400/500
```

Diseño de clases (notación v3)

Cada clase se dibuja con su caja: los miembros **privados** van dentro de la caja y los **públicos** cruzan la pared (`|`). A la izquierda, `-->` apunta al método cuando recibe datos y `<--` sale de él cuando devuelve. A la derecha, `-->` indica que el método **modifica** el estado, `<--` que solo lo **consulta** y `--x` que es **estático** (no usa estado). Entre métodos consecutivos, dos líneas verticales `|`.

`AnalizadorBeacons` (utilidad: todo estático, sin estado)

```
              --------------- AnalizadorBeacons ---------------
              |
              |   COMPANY_ID_APPLE: N
              |   TAM_UUID: N
              |   ETIQUETA_LOG: Text
              |
              |
              |
```

```
                |
resultado: ScanResult --> esNuestroBeacon() --x
                        B <--
                |
                |
bytesTrama: [ N ] --> esNuestroBeacon() --x
                        B <--
                |
                |
bytesTrama: [ N ] --> esTramaIBeacon() --x
                        B <--
                |
                |
resultado: ScanResult --> motivoDescarte() --x
                        Text <--
                |
                |
tib: TramaIBeacon --> interpretarValor() --x
                        Z <--
                |
                |
codigo: Z --> tipoDeMedicion() --x
          Text <--
                |
                |
tib: TramaIBeacon --> tipoDeMedicion() --x
                        Text <--
                |
                |
m: Medicion --> construirJSON() --x
             Text <--
                |
                |
nombre: Text, tipo: Text, valor: Z --> construirJSON() --x
                         Text <--
                |
                |
texto: Text --> escaparTexto() --x
              Text <--
                |
                |
host, puerto, ruta --> construirURL() --x
                       Text <--
                |
                |
ahora: Z, ultimoEnvio: Z --> debeEnviarMedicion() --x
                              B <--
                |
                |
tib: TramaIBeacon --> mostrarTrama() --x
                |
                |
cuerpo: Text --> mostrarJSON() --x
```

```
                |
                -----------------------------------------------
```

`Medicion` **(modelo de dominio)**

```
                    ---------------------- Medicion ---------------------
                    |
                    | nombre: Text
                    | tipoMedicion: Text
                    | valorMedicion: Z
                    |
                    |
tib: TramaIBeacon, nombre: Text --> Medicion() -->
                    |
                    |
Text <-- getNombre() <--
                    |
                    |
Text <-- getTipoMedicion() <--
                    |
                    |
Z    <-- getValorMedicion() <--
                    |
                    ------------------------------------------------------
```

`TramaIBeacon` **(interpreta una lectura BLE)**

```
                    ------------------- TramaIBeacon -------------------
                    |
                    | TIPO_MANUFACTURER: N
                    | COMPANY_ID_APPLE: N
                    | TAM_DATOS_IBEACON: N
                    |
                    | prefijo, uuid, major, minor, losBytes,
                    | advFlags, advHeader, companyID: [ N ]
                    | txPower, iBeaconType, iBeaconLength: N
                    |
                    |
bytes: [ N ] --> TramaIBeacon() -->
                    |
                    |
[ N ]_9  <-- getPrefijo() <--
                    |
                    |
[ N ]_16 <-- getUUID() <--
                    |
                    |
[ N ]_2  <-- getMajor() <--
```

```
                  |
                  |
[ N ]_2  <-- getMinor() <--
                  |
                  |
N        <-- getTxPower() <--
                  |
                  |
[ N ]    <-- getLosBytes() <--
                  |
                  |
[ N ]_3  <-- getAdvFlags() <--
                  |
                  |
[ N ]_2  <-- getAdvHeader() <--
                  |
                  |
[ N ]_2  <-- getCompanyID() <--
                  |
                  |
N        <-- getiBeaconType() <--
                  |
                  |
N        <-- getiBeaconLength() <--
                  |
                  |
bytes: [ N ] --> buscarEstructuraIBeacon() --x
                               Z <--
                  |
                  |
bytes: [ N ] --> esTramaIBeacon() --x
                               B <--
                  |
                  ---------------------------------------------------
```

`Utilidades` **(utilidad: todo estático)**

```
                  ---------------------- Utilidades ----------------------
                  |
                  |
texto: Text --> stringToBytes() --x
                [ Z ] <--
                  |
                  |
uuid: Text --> stringToUUID() --x
               UUID <--
                  |
                  |
uuid: UUID --> uuidToString() --x
                  Text <--
                  |
```

```
                  |
uuid: UUID --> uuidToHexString() --x
                    Text <--
                  |
                  |
bytes: [ Z ] --> bytesToString() --x
                     Text <--
                  |
                  |
mas: N, menos: N --> dosLongToBytes() --x
                           [ Z ]_16 <--
                  |
                  |
bytes: [ Z ] --> bytesToInt() --x
                      Z <--
                  |
                  |
b: Z --> leerByteConSigno() --x
              Z <--
                  |
                  |
bytes: [ Z ] --> bytesToLong() --x
                       Z <--
                  |
                  |
bytes: [ Z ] --> bytesToIntOK() --x
                       Z <--
                  |
                  |
bytes: [ Z ] --> bytesToHexString() --x
                     Text <--
                  |
                  ------------------------------------------------------
```

`LogicaFakeApp` **(lógica de cliente / proxy)**

```
                  -------------------- LogicaFakeApp --------------------
                  |
                  | METODO_POST: Text
                  |
                  | CLASE_VALIDACION: Text
                  | CLASE_PERSISTENCIA: Text
                  |
                  |
m: Medicion, respuesta: RespuestaApp --> guardarMedicion() -->
                  |
                  |
                  |
m: Medicion --> construirPeticion() --x
              Peticion <--
                  |
```

```
                    |
codigo: Z, cuerpo: Text --> interpretarRespuesta() --x
                          ResultadoGuardado <--
                    |
                    |
cuerpo: Text --> leerError() --x
                     Text <--
                    |
                    |
respuesta, resultado --> avisar() --x
                    |
                    |
                    --> crearPeticionarioREST() --x
                              PeticionarioREST <--
                    |
                    ------------------------------------------------------
```

`PeticionarioREST` **(extiende** `AsyncTask`**)**

```
                    -------------------- PeticionarioREST --------------------
                    |
                    | elMetodo: Text
                    | urlDestino: Text
                    | elCuerpo: Text
                    | laRespuesta: RespuestaREST
                    | codigoRespuesta: Z
                    | cuerpoRespuesta: Text
                    |
                    |
codigo: Z, cuerpo: Text --> callback() -->
                    |
                    |
                    --> PeticionarioREST() -->
                    |
                    |
metodo, url, cuerpo, respuesta --> hacerPeticionREST() -->
                    |
                    |
                    --> doInBackground() -->
                             B <--
                    |
                    |
comoFue: B --> onPostExecute() -->
                    |
                    ------------------------------------------------------
```

`ServicioEscuharBeacons` **(extiende** `IntentService`**)**

```
                ---------------- ServicioEscuharBeacons ----------------
                |
                | ETIQUETA_LOG: Text
                | observadorDeTrama: ObservadorDeTrama
                |
                | respuestaDelGuardado: RespuestaApp
                | logicaFakeApp: LogicaFakeApp
                | tiempoDeEspera: N
                | instanteUltimoEnvio: N
                | seguir: B
                | elEscanner: BluetoothLeScanner
                | callbackDelEscaneo: ScanCallback
                |
                |
texto: Text --> mostrarEnPantalla() -->
                |
                |
                --> ServicioEscuharBeacons() -->
                |
                |
observador: ObservadorDeTrama --> setObservadorDeTrama() --x
                |
                |
                --> parar() -->
                |
                |
                --> onDestroy() -->
                |
                |
intencion: Intencion --> onHandleIntent() -->
                |
                |
resultado: ScanResult --> manejarBeaconRecibido() -->
                |
                |
texto: Text --> mostrarEnPantalla() -->
                |
                |
                --> crearLogicaApp() --x
                       LogicaFakeApp <--
                |
                |
                --> iniciarEscuchaBTLE() -->
                |
                |
                --> detenerEscuchaBTLE() -->
                |
                |
elAdaptador: BluetoothAdapter --> estaEncendido() --x
                                        B <--
                |
                |
```

```
                  --> obtenerAdaptador() --x
                      BluetoothAdapter <--
                  |
                  ------------------------------------------------------
```

`MainActivity` **(extiende** `AppCompatActivity`**)**

```
                  -------------------- MainActivity --------------------
                  |
                  | ETIQUETA_LOG: Text
                  | CODIGO_PETICION_PERMISOS: N
                  | PERMISOS_NECESARIOS: [ Text ]
                  | METODO_GET: Text
                  |
                  | elEscanner: BluetoothLeScanner
                  | callbackDelEscaneo: ScanCallback
                  | contadorResultados: Z
                  | elIntentDelServicio: Intent
                  | textoEstadoPermisos, textoUltimaTrama, textoRespuestaRest:
TextView
                  | observador: ObservadorDeTrama
                  |
                  |
v: View --> botonArrancarServicioPulsado() -->
                  |
                  |
v: View --> botonDetenerServicioPulsado() -->
                  |
                  |
v: View --> botonBuscarDispositivosBTLEPulsado() -->
                  |
                  |
v: View --> botonBuscarNuestroDispositivoBTLEPulsado() -->
                  |
                  |
v: View --> botonDetenerBusquedaDispositivosBTLEPulsado() -->
                  |
                  |
v: View --> botonProbarRestPulsado() -->
                  |
                  |
estado: EstadoActividad --> onCreate() -->
                  |
                  |
                  --> onDestroy() -->
                  |
                  |
requestCode, permisos, concesiones --> onRequestPermissionsResult() -->
                  |
                  |
                  --> buscarTodosLosDispositivosBTLE() -->
```

```
                  |
                  |
nombre: Text --> buscarEsteDispositivoBTLE() -->
                  |
                  |
                  --> detenerBusquedaDispositivosBTLE() -->
                  |
                  |
resultado: ScanResult --> mostrarInformacionDispositivoBTLE() <--
                  |
                  |
dispositivo: BluetoothDevice --> nombreSeguro() <-- Text
                  |
                  |
dispositivo: BluetoothDevice --> direccionSegura() <-- Text
                  |
                  |
                  --> pidePermisosSiFaltan() -->
                           B <--
                  |
                  |
                  --> faltanPermisos() --x
                           B <--
                  |
                  |
                  --> obtenerEscaner() --x
                         BluetoothLeScanner <--
                  |
                  |
                  --> estaElBluetoothEncendido() --x
                           B <--
                  |
                  ------------------------------------------------------
```

Design Clarifications

- `LogicaFakeApp` es la **lógica de cliente** de la app: por la **regla del subconjunto** solo declara `guardarMedicion()` (la app solo necesita enviar), y por la **regla del proxy** encapsula toda la comunicación con el servidor.
- El **servicio BLE** (`ServicioEscuharBeacons`) no sabe de JSON, direcciones ni códigos de estado: solo llama a `guardarMedicion()` y recibe un `ResultadoGuardado`. La dependencia se inyecta por `crearLogicaApp()` (método sobrescribible en pruebas).
- Las funciones `construirPeticion()`, `interpretarRespuesta()` y `leerError()` son **detalles de implementación** (no forman parte del contrato); son puras, no hacen red y se pueden probar en el PC.
- El **throttle** (`debeEnviarMedicion`) sigue en el servicio, no en el proxy: es una política de envío del escaneo, no una regla de guardado. El backend tampoco limita la frecuencia.
- El resultado se expresa siempre con **tipos de dominio** (`ok`, `error`, `clase`); la traducción desde el código HTTP es un detalle interno del proxy.
- La app **no** guarda mediciones localmente: cada medición se sube al servidor.